

Manually triggered skills via `/name` (CC 2.1.3)

CC 2.1.3: Skills use `.claude/skills/<name>/SKILL.md` ; set `disable-model-invocation: true` for manual invocation only. Legacy `.claude/commands/*.md` files remain supported. The `/name` syntax and behavior are identical.

Principle

A slash command is a Markdown file that defines a workflow. Claude executes it as if it had received that text as a prompt, with the ability to pass dynamic arguments.

File locations

```
.claude/skills/          # Project skills
├── commit/SKILL.md      → /commit
├── review-pr/SKILL.md   → /review-pr
└── deploy/SKILL.md      → /deploy

~/.claude/skills/        # Personal skills
├── release/SKILL.md     → /release
└── sync/SKILL.md        → /sync
```

Global commands are available in all sessions, regardless of the project.

Invocation and arguments

```
/commit          # No argument
/deploy production # Positional argument
/release minor    # Version bump
```

In the Markdown file, arguments are accessible via variables:

```
Deploy to environment: $ARGUMENTS[0]
With strategy: $ARGUMENTS[1]

If no argument provided: use "staging" by default.
```

The `$0` , `$1` syntax is equivalent to `$ARGUMENTS[0]` , `$ARGUMENTS[1]` . The old dot notation (`$ARGUMENTS.0`) is deprecated since v2.1.19.

Recommended structure

```
# Command Name

## Objective
What this command does in one sentence.

## Process
1. Step 1 – detailed instructions
2. Step 2 – detailed instructions
3. Step 3 – detailed instructions

## Arguments
- $0: target environment (default: staging)

## Output format
Description of the expected result.
```

Recent native commands

Command	Usage
<code>/goal [condition]</code>	Autonomous multi-turn mode: Claude works until the condition is met, with a live elapsed/turns/tokens overlay (v2.1.139)
<code>/scroll-speed</code>	Interactive slider to tune mouse wheel scroll speed with live preview (v2.1.139)

Useful commands to create

Command	Usage
<code>/commit</code>	Automatic conventional commit
<code>/review-pr</code>	PR review against project criteria
<code>/release [patch minor major]</code>	Version bump + changelog
<code>/sync</code>	Multi-file consistency check
<code>/security-check</code>	Quick config scan

Command vs skill vs agent

Mechanism	When to use
Command	One-off workflow, procedure to follow
Skill	Reusable knowledge + embedded resources
Agent	Recurring specialist with own memory

A skill directory can include additional reference files and scripts, whether invoked by the user or model. Agent memory is a separate persistent mechanism.

Practical tips

One command, one responsibility. As soon as a command does two distinct things, split it. Name with an action verb first (`commit` , `release` , `sync`) so that the intent is immediately readable in the list of available commands.